

LEVERAGING PREDICTABILITY OF DEEP LEARNING TECHNIQUES ON FINANCIAL TIME SERIES FORECASTING

**A Project Report submitted in partial fulfillment of the requirements for the award
of the degree of**

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

Submitted by

SRI VAISHNAVI VASA - HU21CSEN0100673

LEENA DEEKOLLU - HU21CSEN0100741

Y. BINDU SRI - HU21CSEN0101184

Under the esteemed guidance of

Dr. Sarat Chandra Nayak

Professor



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

GITAM (Deemed to be University)

HYDERABAD

OCTOBER 2024

**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
GITAM SCHOOL OF TECHNOLOGY
GITAM (Deemed to be University)**



DECLARATION

I/We, hereby declare that the project report entitled “**Leveraging predictability of deep learning techniques on financial time series forecasting**” is an original work done in the Department of Computer Science and Engineering, GITAM School of Technology, GITAM (Deemed to be University) submitted in partial fulfillment of the requirements for the award of the degree of B.Tech. in Computer Science and Engineering. The work has not been submitted to any other college or University for the award of any degree or diploma.

Date: 28th October 2024

Registration No(s)	Name(s)	Signature(s)
HU21CSEN0100673	Sri Vaishnavi Vasa	
HU21CSEN0100741	Leena Deekollu	
HU21CSEN0101184	Y. Bindu Sri	

**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
GITAM SCHOOL OF TECHNOLOGY
GITAM (Deemed to be University)**



CERTIFICATE

This is to certify that the project report entitled “**Leveraging predictability of deep learning techniques on financial time series forecasting**” is a bonafide record of the work carried out by **HU21CSEN0100673, HU21CSEN0100741, and HU21CSEN0101184**, students of the Bachelors of Technology program in Computer Science & Engineering. This report is submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering.

Project Guide

Prof. Sarat Chandra Nayak
Professor
Dept of CSE

Project Coordinator

Dr. A B Pradeep Kumar
Assistant Professor
Dept of CSE

Head of the Department

Prof. Shaik Mahaboob Basha
Professor
Dept of CSE

ACKNOWLEDGEMENT

Our project report would not have been successful without the help of several people. We would like to thank the personalities who were part of our seminar in numerous ways, those who gave us outstanding support from the birth of the seminar.

We are extremely thankful to our honorable Pro-Vice-Chancellor, **Prof. D. Sambasiva Rao**, for providing the necessary infrastructure and resources for the accomplishment of our seminar. We are highly indebted to **Prof. N. Seetharamaiah**, Associate Director, School of Technology, for his support during the tenure of the seminar.

We are very much obliged to our beloved **Prof. Mahaboob Basha**, Head of the Department of Computer Science & Engineering, for providing the opportunity to undertake this seminar and encouragement in the completion of this seminar. **Prof. Sarat Chandra Nayak**, Professor, Department of Computer Science and Engineering, School of Technology for the esteemed guidance, moral support and invaluable advice provided by them for the success of the project report.

We are also thankful to all the Computer Science and Engineering department staff members who have cooperated in making our seminar a success. We would like to thank all our parents and friends who extended their help, encouragement, and moral support directly or indirectly in our seminar work.

Sincerely,

ABSTRACT

Financial time series forecasting has become a vital focus within computational intelligence, engaging researchers in academia and industry alike due to its broad applications and substantial impact. While numerous machine learning (ML) models have been developed for this purpose, the advent of deep learning (DL) has significantly enhanced forecasting accuracy by capturing complex, nonlinear patterns in financial data. However, a notable gap exists in region-specific research, particularly for financial markets like the Bombay Stock Exchange (BSE).

This project addresses this gap by systematically reviewing existing DL models and experimenting with architectures such as CNNs, RNNs, and LSTMs to evaluate their effectiveness for the BSE dataset. Our work extends beyond evaluation to identify and optimize the best-performing model while introducing a novel DL architecture tailored specifically to Indian stock market dynamics. By addressing challenges such as nonstationarity, overfitting, and interpretability, this research aims to advance the predictive capabilities of DL in financial forecasting.

The findings from this study will contribute to the development of robust, accurate forecasting models, offering insights valuable to both academia and industry. Moreover, the novel approach proposed in this work has the potential to set a new benchmark for financial time series forecasting, particularly in region-specific contexts.

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION.....	7
1.1 PROBLEM DEFINITION.....	7
1.2 OBJECTIVE.....	8
1.3 LIMITATIONS.....	8
1.4 OUTCOMES.....	8
1.5 APPLICATIONS.....	9
CHAPTER 2: LITERATURE REVIEW.....	9
2.1 TRADITIONAL STATISTICAL MODELS.....	9
2.2 MACHINE LEARNING APPROACHES.....	10
2.3 DEEP LEARNING MODELS.....	10
2.3.1 Recurrent Neural Networks (RNNs) and Variants.....	10
2.3.2 Convolutional Neural Networks (CNNs).....	11
2.3.3 Attention Mechanisms and Transformers.....	11
2.3.4 Emerging Architectures.....	11
CHAPTER 3: PROBLEM ANALYSIS.....	11
3.1 PROBLEM STATEMENT.....	12
3.2 EXISTING SYSTEM.....	12
3.3 FLAWS AND DISADVANTAGES.....	13
3.4 PROPOSED SYSTEM.....	14
3.5 FUNCTIONAL REQUIREMENTS.....	15
3.6 NON-FUNCTIONAL REQUIREMENTS.....	15
CHAPTER 4: IMPLEMENTATION.....	16
4.1 OVERVIEW OF TECHNOLOGIES.....	16
4.1.1 Python.....	16
4.1.2 Google Colab.....	16
4.1.3 Git.....	16
4.1.4 Jupyter Notebook.....	16
4.2 WORKFLOW.....	17
4.3 LIBRARIES IMPORTED.....	17
4.3.1 TensorFlow.....	17
4.3.2 Numpy.....	17
4.3.3 Pandas.....	17
4.3.4 Scikit-Learn.....	18
5.4 DATASET.....	18
4.4.1 Dataset Analysis.....	18

4.4.2 Bombay Stock Exchange SENSEX NEXT50.....	21
4.4.3 Annotations.....	21
4.5 ALGORITHMS IMPLEMENTED.....	21
CHAPTER 5: TESTING AND VALIDATION.....	27
5.1 TEST PARAMETERS.....	27
5.2 PERFORMANCE METRICS OF ARIMA.....	28
5.3 PERFORMANCE OF SARIMA.....	30
5.4 PERFORMANCE OF CNN.....	31
5.4.1 Performance of CNN 1D.....	31
5.4.2 Performance of CNN 2D.....	32
5.4.3 Performance of CNN 3D.....	33
5.5 PERFORMANCE OF RNN.....	34
5.6 PERFORMANCE OF LSTM.....	35
CHAPTER 6: RESULT ANALYSIS.....	35
6.1 RESULT SUMMARY OF ARIMA MODEL.....	35
6.2 RESULT SUMMARY OF SARIMA MODEL.....	36
6.3 RESULT SUMMARY OF CNN MODEL.....	37
6.3.1 CNN 1D Model.....	37
6.3.2 CNN 2D Model.....	37
6.3.3 CNN 3D Model.....	38
6.4 RESULT SUMMARY OF RNN MODEL.....	39
6.5 RESULT SUMMARY OF LSTM MODEL.....	40
CHAPTER 7: CONCLUSION.....	41
7.1 PROPOSED MODEL.....	41
7.2 FUTURE SCOPE/GOALS.....	42
REFERENCES.....	43

CHAPTER 1: INTRODUCTION

Financial time series forecasting is a critical area within computational intelligence, heavily influencing decision-making processes in trading, risk management, and portfolio optimization. With the evolution of machine learning (ML) methods, traditional statistical models like ARIMA and GARCH have been increasingly complemented by modern computational techniques. Deep learning (DL), with its ability to model complex, non-linear relationships, has emerged as a promising approach to address the limitations of earlier methods.

In recent years, DL models such as LSTMs, CNNs, and attention-based architectures have achieved significant success in time series prediction. However, these advancements have primarily focused on generalized datasets or specific global markets, often overlooking the nuances of region-specific financial systems. For example, the Bombay Stock Exchange (BSE), with its unique patterns influenced by local economic and regulatory factors, presents challenges and opportunities for financial forecasting models.

This research seeks to bridge this gap by exploring the potential of DL techniques tailored for Indian stock data. By leveraging advanced methodologies and addressing inherent challenges like data nonstationarity and volatility, this study aims to enhance forecasting accuracy and provide actionable insights for market participants.

The subsequent sections of this chapter provide a detailed view of the problem definition, objectives, limitations, anticipated outcomes, and potential applications of the study. These elements collectively frame the foundation for this research, highlighting its relevance and expected contributions.

1.1 PROBLEM DEFINITION

Forecasting financial time series, such as stock prices, is a complex yet essential task in financial analytics. Traditional models often struggle with capturing the non-linearity, volatility, and sudden market shifts inherent in financial data. This is exacerbated by challenges like missing data, nonstationarity due to market regulations or global events, and overfitting in predictive models. While deep learning (DL) techniques have demonstrated potential in addressing these issues, their application remains limited in region-specific contexts like the Bombay Stock Exchange (BSE). Existing research lacks sufficient focus on DL methodologies optimized for Indian financial data, creating a gap that this study aims to address.

1.2 OBJECTIVE

The primary objective of this research is to develop a deep learning model optimized for financial time series forecasting on the Bombay Stock Exchange. This involves:

- Experimenting with advanced DL models (e.g., LSTM, CNN, hybrid architectures) to enhance predictive accuracy.
- Addressing key challenges such as overfitting, data nonstationarity, and computational efficiency.
- Reviewing state-of-the-art DL methodologies and identifying opportunities for novel improvements.

Ultimately, this study aims to propose a model that surpasses existing benchmarks and provides actionable insights into market trends.

1.3 LIMITATIONS

This research faces certain limitations that may affect its scope:

1. Data Availability: The quality and quantity of region-specific data, particularly for BSE, may limit model training and evaluation.
2. Computational Complexity: DL models often require significant computational resources, which may pose constraints during extensive hyperparameter tuning or ensemble modeling.
3. Model Generalizability: Models optimized for the BSE might not generalize well to other markets or financial systems.
4. Real-Time Implementation: The study focuses on forecasting accuracy rather than real-time deployment, which may require additional considerations for latency and scalability.

1.4 OUTCOMES

The expected outcomes of this research include:

1. Development of a region-specific DL model optimized for BSE financial data.
2. Improved forecasting accuracy compared to traditional statistical models and existing DL techniques.
3. Insights into the effectiveness of various DL architectures (e.g., LSTM, CNN) for addressing financial data challenges like nonstationarity and volatility.
4. A comprehensive review of current DL methodologies, highlighting their strengths and limitations in financial forecasting.

1.5 APPLICATIONS

The results of this study hold significant potential for applications in various financial domains:

1. Trading Strategies: Enhanced forecasting accuracy can aid traders in making informed decisions on stock movements.
2. Risk Management: Improved predictions of market volatility can help financial institutions mitigate risks effectively.
3. Portfolio Optimization: Accurate forecasts can support asset allocation strategies, maximizing returns while minimizing risks.
4. Policy Development: Insights from this study may assist regulators in understanding market behavior and implementing effective policies.
5. Regional Market Research: The tailored focus on BSE makes this research valuable for local market participants and analysts.

CHAPTER 2: LITERATURE REVIEW

The literature review aims to provide a comprehensive understanding of the advancements, challenges, and gaps in financial time series forecasting, focusing particularly on the application of deep learning (DL) techniques. For this study, a total of 25 research papers were reviewed. Among these, 18 were focused on deep learning models, while the remaining 7 explored traditional statistical methods, hybrid models, or alternative machine learning techniques.

This section is organized into subchapters for clarity, categorizing the reviewed works based on their methodologies and contributions.

2.1 TRADITIONAL STATISTICAL MODELS

Statistical models such as ARIMA, GARCH, and their variants have been the cornerstone of financial forecasting for decades. Among the reviewed papers, 7 emphasized these methods, highlighting their strengths and limitations.

- ARIMA and GARCH Models: ARIMA remains a widely used baseline model due to its simplicity and effectiveness in capturing linear trends. However, its inability to handle nonstationarity and nonlinearity in financial data is a notable limitation (Cheng et al., 2021). GARCH models, often used for volatility forecasting, perform well in capturing

time-varying variance but struggle with long-term dependencies (Mutinda & Langat, 2024).

- Hybrid Models: A few studies proposed combining ARIMA or GARCH with machine learning models to overcome their individual limitations. For example, the integration of GARCH with AI models like LSTM and GRU demonstrated improved accuracy, especially for highly volatile data (Mutinda & Langat, 2024).

2.2 MACHINE LEARNING APPROACHES

The transition to machine learning (ML) introduced models like Support Vector Machines (SVM), Random Forest (RF), and Gradient Boosted Decision Trees (GBDT). These models were commonly used as benchmarks in the reviewed papers. Key insights include:

- Advantages: ML models are better equipped to handle nonlinearity than statistical models (Zhao et al., 2024).
- Challenges: They often require extensive feature engineering and may not scale effectively for high-frequency or multivariate data (Zhao et al., 2024).
- Techniques: Papers utilizing ML models emphasized hyperparameter tuning techniques, such as GridSearch and Bayesian Optimization, to improve performance (Zhao et al., 2024).

2.3 DEEP LEARNING MODELS

Deep learning emerged as the dominant focus, with 18 of the reviewed papers exploring various architectures.

2.3.1 Recurrent Neural Networks (RNNs) and Variants

- LSTM and GRU: These architectures excel in capturing long-term dependencies and non-linear patterns, making them a popular choice for financial forecasting (Zhang et al., 2024). A study utilizing a Dual-LSTM model achieved superior results by addressing semi-convergence issues during training (Gajamannage et al., 2023).
- Bi-Directional LSTMs: Papers emphasized their ability to model forward and backward dependencies, which is particularly useful in volatile financial environments (Gajamannage et al., 2023).

2.3.2 Convolutional Neural Networks (CNNs)

- 1D, 2D, and 3D CNNs: These architectures were applied to extract spatial features from financial data, such as price trends. While CNNs showed promise, their performance often depended on the quality of preprocessed input data (Salgotra et al., 2024).
- Hybrid CNN-RNN Models: Some studies combined CNNs for feature extraction with RNNs for sequential modeling, yielding significant performance gains (Salgotra et al., 2024).

2.3.3 Attention Mechanisms and Transformers

- Papers exploring attention mechanisms highlighted their ability to focus on relevant time periods in the input data, improving interpretability and accuracy (Alonso & Franklin, 2024).
- Transformer models, initially developed for NLP, were adapted for time series forecasting. While their performance was promising, the high computational cost remained a concern (Alonso & Franklin, 2024).

2.3.4 Emerging Architectures

- Graph Neural Networks (GNNs): Demonstrated an advantage in capturing relationships between entities (e.g., stocks and events), improving forecasting accuracy (Cheng et al., 2021).
- Generative Adversarial Networks (GANs): Showed potential for generating synthetic financial data for training, addressing data scarcity issues (Cheng et al., 2021).

This review highlights the diverse methodologies applied in financial forecasting, revealing opportunities to integrate advanced architectures, such as attention mechanisms and hybrid models, for improved performance.

CHAPTER 3: PROBLEM ANALYSIS

Forecasting financial time series data presents significant challenges, primarily due to market volatility, nonstationarity, noise, and gaps in historical records. While deep learning (DL) models, such as LSTMs, GRUs, and CNNs, have made notable progress in time series prediction, they often lack the ability to generalize across different datasets. This research aims

to address these limitations by developing a robust and adaptable forecasting solution specifically tailored to the unique characteristics of Bombay Stock Exchange (BSE) data.

3.1 PROBLEM STATEMENT

Financial time series forecasting is inherently challenging due to several factors:

- Nonstationarity: Financial data often exhibit structural breaks caused by economic or geopolitical events, making the data nonstationary. This makes it difficult for traditional models like ARIMA to perform well.
- Volatility and Complexity: Financial markets are highly volatile and often chaotic, with nonlinear relationships between variables. Models need to capture these intricate dependencies to provide accurate forecasts.
- Data Gaps and Noise: Missing data and noisy records are common in financial datasets, further complicating model training and forecasting.
- Limited Generalization: Many existing models are designed for global datasets, and their performance significantly deteriorates when applied to regional datasets like those of the BSE.
- Overfitting and Scarcity of Data: High model complexity and limited regional data can lead to overfitting, diminishing a model's ability to generalize to unseen data.

This study proposes a deep learning-based solution aimed at overcoming these challenges, focusing on a hybrid model combining CNN 3D for capturing spatial and temporal dependencies, alongside other advanced architectures like LSTMs for sequential pattern modeling. This model will be designed to optimize forecasting accuracy specifically for Indian stock data from the BSE.

3.2 EXISTING SYSTEM

Existing systems for financial time series forecasting generally rely on statistical methods, machine learning (ML) models, and deep learning (DL) approaches. However, they suffer from key limitations when applied to datasets like the BSE:

1. **Statistical Models**:
 - ARIMA, GARCH, SARIMA: While effective for stationary time series, these models struggle to capture the nonlinear dynamics and nonstationarity inherent in financial markets. They also fail to model the complex interdependencies between multiple financial variables.

- Limitations: These models are rigid and lack flexibility when applied to nonstationary or high-dimensional datasets, and their performance deteriorates in volatile market conditions.

2. Machine Learning Models:

- Random Forests, Support Vector Machines (SVMs), Gradient Boosting: These models improve on feature extraction compared to statistical models. However, they do not fully capture the sequential nature of financial time series and often lack the ability to handle the complexity of dynamic market conditions.
- Limitations: ML models often fail to consider temporal dependencies and do not provide a means for capturing long-term relationships in financial data.

3. Deep Learning Models:

- Recurrent Neural Networks (RNNs), Long Short-Term Memory (LSTM), and Convolutional Neural Networks (CNNs): DL models are capable of capturing sequential dependencies and nonlinearities. However, initial models like LSTMs and vanilla CNNs struggle to effectively model both temporal and spatial features in financial data.
- Limitations: Most existing DL models for financial forecasting are designed for global markets, and their effectiveness decreases when applied to regional datasets like the BSE. Additionally, these models face challenges with overfitting, especially when data is noisy or sparse.

Despite the progress in DL-based forecasting, current systems for BSE forecasting lack the ability to handle region-specific characteristics, external variables, and the complex nature of financial data effectively.

3.3 FLAWS AND DISADVANTAGES

Based on insights from existing research and industry practices, several key limitations of current financial forecasting systems are identified:

1. Generalization Issues:
DL models trained on global datasets or large-scale stock markets often perform poorly when applied to region-specific markets like the BSE, which have different market dynamics, regulations, and trading patterns.
2. Overfitting and Data Scarcity:
Financial datasets, especially for smaller or region-specific markets like the BSE, often have limited data availability. High model complexity, in the absence of sufficient data, leads to overfitting, reducing the model's ability to generalize effectively.
3. Lack of Interpretability:

DL models, particularly deep neural networks, are often considered "black-box" models. This lack of transparency makes it difficult for analysts and stakeholders to trust the model's predictions, especially in high-stakes financial decision-making.

4. Exclusion of External Data:

Macroeconomic indicators, sentiment analysis from news, and geopolitical events play a significant role in stock market performance but are often excluded from forecasting models. The failure to integrate these external variables limits the model's predictive accuracy.

5. Limited Adaptability:

Current models are typically designed for global datasets, and their application to local datasets like the BSE requires significant modifications. As a result, they fail to adapt to the unique characteristics and behaviors of regional financial markets.

3.4 PROPOSED SYSTEM

The proposed system utilizes CNN 3D as the core architecture for capturing both spatial and temporal patterns in financial time series data, making it suitable for forecasting on the Bombay Stock Exchange (BSE). CNN 3D is well-suited to handle the multidimensional and sequential nature of financial data, optimizing predictive performance for market trends.

1. CNN 3D Architecture:

Specifically designed to capture spatiotemporal dependencies in financial data, maintaining temporal coherence across multidimensional time series.

2. Future Scope for Hybrid Models:

While CNN 3D is the focus of the current model, future work will explore integrating LSTM or other advanced models to capture long-term dependencies and enhance forecasting accuracy.

3. Scalability and Adaptability:

The system is tailored for BSE data but is designed to adapt to other markets or datasets in the future.

By leveraging CNN 3D, the proposed system focuses on enhancing accuracy in financial forecasting, with plans to further improve the model by incorporating hybrid approaches in the future.

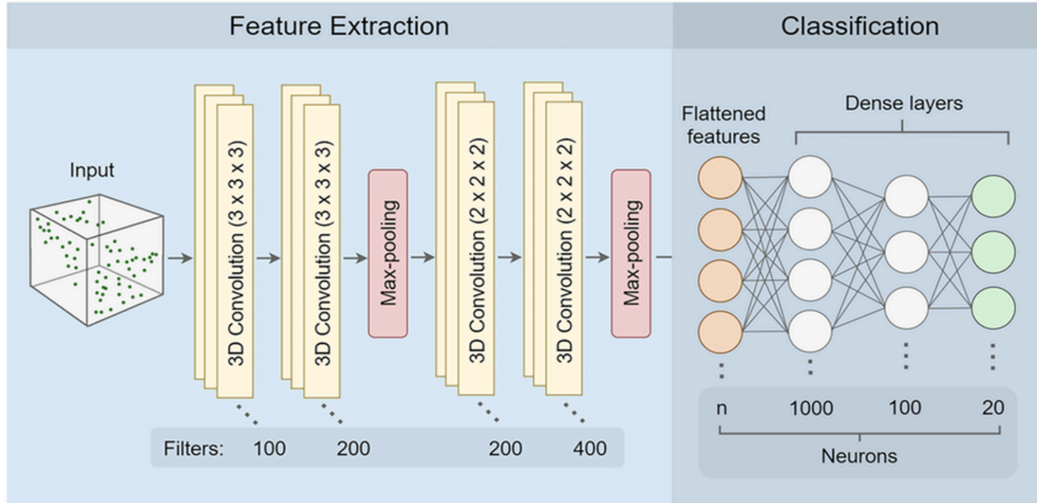


Fig 1: Architecture of CNN 3D model

3.5 FUNCTIONAL REQUIREMENTS

The functional requirements of the system include:

1. Accurate Forecasting: The system should predict stock price movements for the BSE with high accuracy, considering both price prediction and directional forecasting.
2. Data Handling: The system must handle missing data and noise effectively, employing techniques like normalization and outlier detection.
3. External Data Integration: The system should incorporate external variables (e.g., macroeconomic data, sentiment analysis) to enhance predictive capabilities.
4. Real-time Prediction: The system should be capable of providing real-time predictions for actionable financial insights.

3.6 NON-FUNCTIONAL REQUIREMENTS

The non-functional requirements focus on the overall usability and performance of the system:

1. Scalability: The system must be able to scale to handle increasing volumes of data without performance degradation.
2. Robustness: The system should maintain prediction accuracy even in the presence of noisy or incomplete data.
3. Efficiency: The system should be optimized for low-latency predictions, enabling real-time forecasting.
4. Interpretability: The model should provide interpretable outputs to allow users to understand the rationale behind predictions.

5. Portability: The system should be easily adaptable to other financial markets or datasets with minimal modification.

CHAPTER 4: IMPLEMENTATION

4.1 OVERVIEW OF TECHNOLOGIES

4.1.1 Python

Python is the primary programming language used for the implementation of the forecasting model. Its rich ecosystem of libraries and frameworks for data analysis, machine learning, and deep learning makes it ideal for tackling complex tasks like financial time series forecasting. Python's ease of use, combined with its flexibility, makes it an optimal choice for this project.

4.1.2 Google Colab

Google Colab is an essential tool for the development and experimentation of the model. It provides free access to cloud-based GPUs and TPUs, which is particularly helpful for training deep learning models like CNN 3D and LSTM. Google Colab also offers an interactive environment for writing and executing Python code, which facilitates collaboration and experimentation.

4.1.3 Git

Git is used for version control, allowing the development team to track changes in the codebase, manage updates, and collaborate efficiently. By using Git, the project ensures that code changes are systematically documented and reversible, improving team collaboration and the maintainability of the system.

4.1.4 Jupyter Notebook

Jupyter Notebooks are used for interactive development and experimentation. It allows for the easy visualization of data, running code in chunks, and documenting the process with markdown, which is particularly useful for analysis and debugging during the model development phase.

4.2 WORKFLOW

The workflow of the implementation follows these key steps:

1. Data Collection: Downloading the financial dataset from the official **BSE** website.
2. Data Preprocessing: Cleaning the data, handling missing values, and transforming it into a format suitable for analysis.
3. Exploratory Data Analysis (EDA): Conducting various analyses, such as correlation matrices and volatility checks.
4. Model Building: Training different forecasting models, including ARIMA, SARIMA, RNN, CNN 1D, 2D, and 3D, and LSTM.
5. Evaluation: Assessing model performance using appropriate metrics and visualizations.
6. Model Tuning: Fine-tuning hyperparameters to improve accuracy.
7. Final Deployment: Implementing the best-performing model for real-time prediction.

4.3 LIBRARIES IMPORTED

The following libraries were imported to assist with data manipulation, machine learning, and model building:

4.3.1 TensorFlow

TensorFlow is the primary library used for building and training deep learning models, including CNN 3D and LSTM. It provides a comprehensive, flexible ecosystem to develop machine learning models with a focus on neural networks.

4.3.2 Numpy

Numpy is used for handling large multi-dimensional arrays and matrices, making it essential for mathematical operations and data manipulation throughout the project.

4.3.3 Pandas

Pandas is used for data preprocessing and analysis. It provides powerful tools to clean, manipulate, and analyze structured data in the form of DataFrames, which is key for handling time series data.

4.3.4 Scikit-Learn

Scikit-Learn is used for traditional machine learning models like ARIMA, SARIMA, and SVM. It also provides tools for preprocessing, model evaluation, and hyperparameter tuning.

5.4 DATASET

The dataset used for the project is downloaded from the **Bombay Stock Exchange (BSE)** official website and contains historical data related to stock market movements.

4.4.1 Dataset Analysis

- Correlation Matrix: A correlation matrix was generated to check the relationship between the closing price and other variables over different years. This helps identify which variables have strong or weak correlations with the closing price.

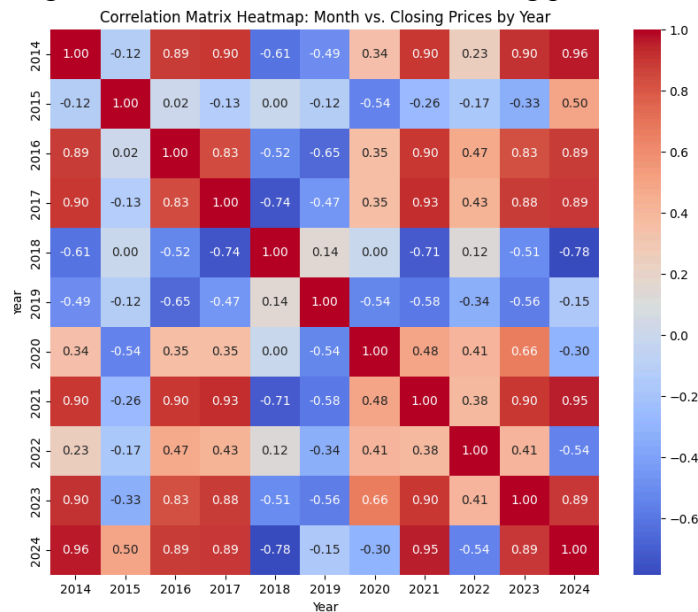


Fig 2: Correlation matrix between Closing price against each year

- Monthly Average Closing Price Comparison: The average closing prices for each month across different years were compared. This comparison helps understand monthly trends and seasonal patterns in stock prices.

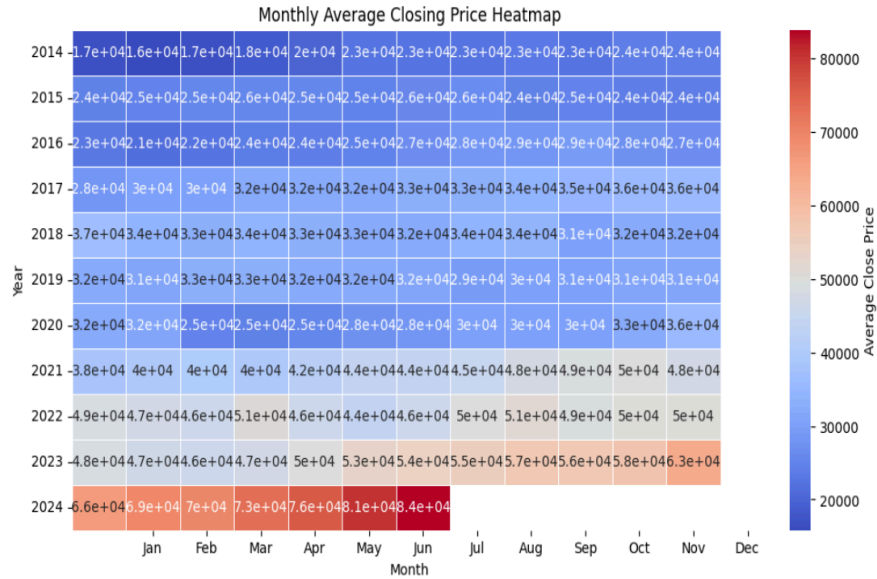


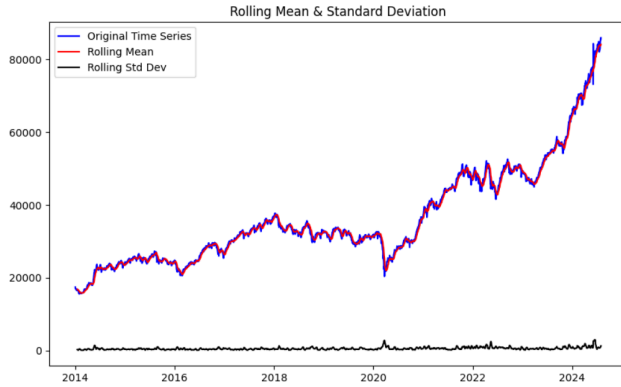
Fig 3: Comparison of monthly average closing price against each year

- **Volatility Analysis:** A graph was generated to illustrate the yearly volatility of the closing prices. High volatility indicates significant price fluctuations, while low volatility suggests relative stability.



Fig 4: The graph illustrates the yearly volatility of the closing prices. (High volatility indicates significant price fluctuations, while low volatility suggests relative stability.)

- **ADF Test (Augmented Dickey-Fuller Test):** The ADF test was applied to check the stationarity of the dataset. Stationarity is crucial for many time series models, and this test helps determine whether the time series data needs differencing or transformation.



ADF Statistic	2.537351
p-Value	0.999061
Critical Values	1%, -3.432856986061305
Critical Values	5%, -2.8626475938238967
Critical Values	10%, -2.5673596115790938

Fig 5: ADF(Augmented Dickey-Fuller) Test on the given data

- Descriptive Statistics: Basic descriptive statistics (mean, median, standard deviation, etc.) were computed to summarize the dataset and identify trends, outliers, and anomalies.

Count	2614
Mean	36585.49759
Variance	188878744.8
Standard Deviation	13743.31637
25th Percentile	26334.85
50th Percentile	32369.58
75th Percentile	45937.805
Min	15574.85
Max	85930.29

Fig 6: Descriptive Statistics for Closing Price

- Time Series Trends: Various trends, including recurring patterns and changes in values, were analyzed using visualization and statistical methods to identify long-term patterns.

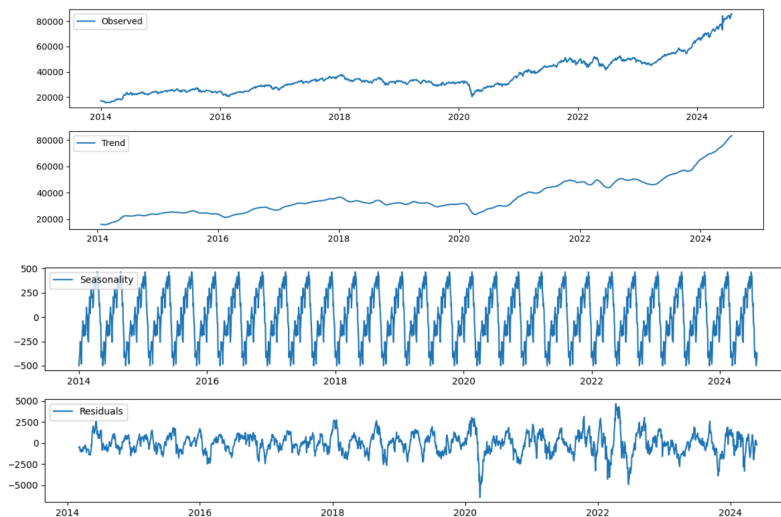


Fig 7: Time Series Trends which includes checking for recurring patterns, change in values etc.

- Savgol Filter: A simple smoothing filter (Savitzky-Golay filter) was applied to the closing price data to observe the long-term trends while removing short-term noise.

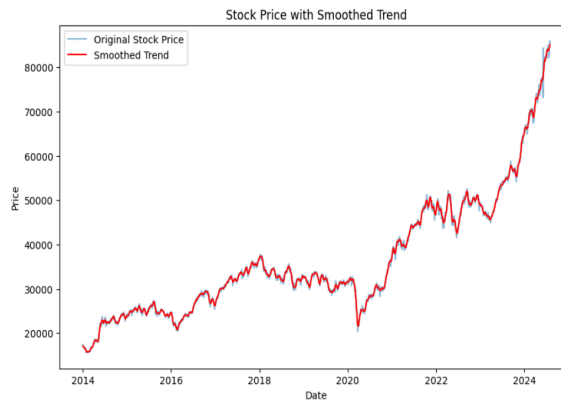


Fig 8: After applying a simple smoothing filter to see the long-term trend (Savgol Filter)

4.4.2 Bombay Stock Exchange SENSEX NEXT50

The SENSEX NEXT50 index from BSE includes the 50 largest companies after the SENSEX 30, representing small- and mid-cap stocks with high liquidity. This index offers a broader view of the Indian market, capturing sectors with significant growth potential. These stocks exhibit higher volatility, which is valuable for forecasting market sentiment and risk. The dataset also includes lower-volume trades, useful for studying less-liquid asset behavior. Additionally, BSE data reflects market responses to major events like policy shifts and crises, providing insights into market resilience. With historical data dating back to 1875, this dataset supports long-term trend analysis.

4.4.3 Annotations

Annotations are added to various graphs and analyses to provide additional context or highlight significant trends, helping to interpret the data more effectively.

4.5 ALGORITHMS IMPLEMENTED

The following algorithms were implemented for time series forecasting and compared to evaluate their performance in predicting stock price movements. Each algorithm was assessed for its ability to handle non-stationary, volatile, and noisy financial data.

1. ARIMA (AutoRegressive Integrated Moving Average)

ARIMA is a popular statistical method used for forecasting stationary time series data. It combines autoregression (AR), differencing (I) to achieve stationarity, and moving averages (MA) to model past values. ARIMA is useful when the data shows linear trends, but it struggles with highly volatile, non-linear, or seasonal data, which makes it less suitable for financial forecasting where trends and patterns are more complex.



Fig 9: Working of ARIMA model

The general ARIMA model equation is:

$$y_t = c + \phi_1 y_{t-1} + \phi_2 y_{t-2} + \dots + \phi_p y_{t-p} + \theta_1 \epsilon_{t-1} + \theta_2 \epsilon_{t-2} + \dots + \theta_q \epsilon_{t-q} + \epsilon_t$$

Where:

- y_t is the time series value at time t
- ϕ are the AR coefficients
- θ are the MA coefficients
- ϵ_t is white noise (error term)
- d denotes differencing to make the time series stationary.

Fig 10: Equation for ARIMA model

2. SARIMA (Seasonal ARIMA)

SARIMA extends ARIMA by incorporating seasonal components, making it better suited for data with seasonality or cyclical behavior, such as monthly or quarterly patterns. It includes seasonal autoregressive (SAR), seasonal differencing (SD), and seasonal moving average (SMA) terms. While SARIMA performs better than ARIMA for seasonal data, it still falls short in capturing non-linear relationships, making it less effective for financial time series data with complex interactions.

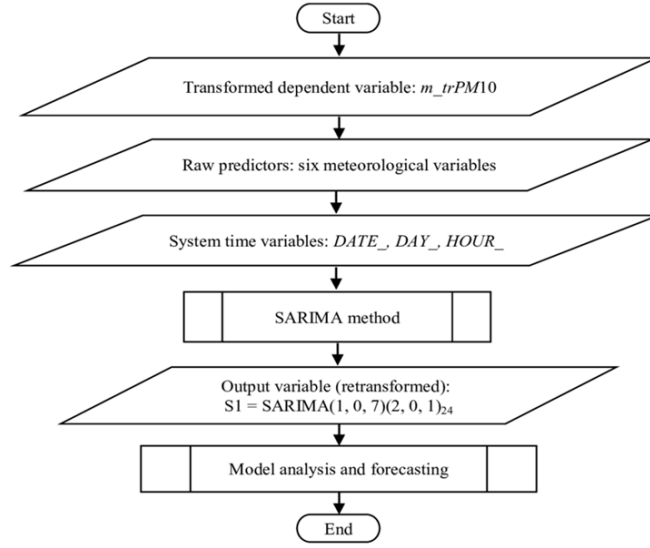


Fig 11: Flowchart of working of SARIMA

The general SARIMA equation is:

$$y_t = c + \sum_{i=1}^p \phi_i y_{t-i} + \sum_{i=1}^q \theta_i \epsilon_{t-i} + \sum_{i=1}^P \Phi_i y_{t-is} + \sum_{i=1}^Q \Theta_i \epsilon_{t-is} + \epsilon_t$$

Where:

- Φ_i and Θ_i are seasonal AR and MA coefficients
- s denotes the seasonal period (e.g., 12 for monthly data with yearly seasonality).

Fig 12: Equation of SARIMA

3. RNN (Recurrent Neural Networks)

RNNs are designed for sequential data, where outputs depend on previous inputs. They capture temporal dependencies by processing sequences step by step, making them suitable for time series forecasting. However, RNNs struggle with long-term dependencies due to vanishing gradients and may fail to capture complex patterns or learn from large amounts of data in volatile markets.

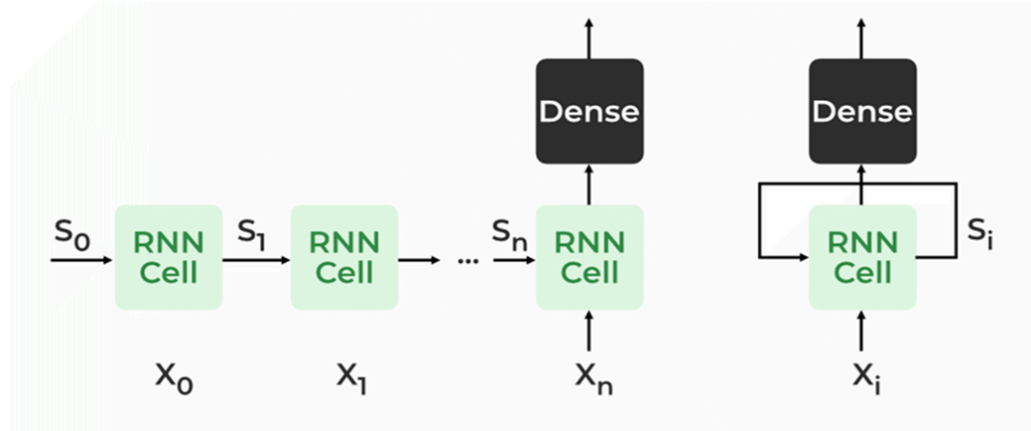


Fig 13: Architecture of RNN

$$h_t = \tanh(W_{xh} \cdot x_t + W_{hh} \cdot h_{t-1} + b_h)$$

where:

- h_t is the hidden state at time t ,
- W_{xh} is the weight matrix for the input x_t ,
- W_{hh} is the weight matrix for the previous hidden state h_{t-1} ,
- b_h is the bias vector,
- $\tanh$ is the activation function applied element-wise.

If an output y_t is needed at each time step, it can be calculated as:

$$y_t = W_{hy} \cdot h_t + b_y$$

Fig 14: Equation of RNN model

4. CNN 1D, 2D, and 3D (Convolutional Neural Networks)

Convolutional Neural Networks (CNNs) are primarily known for image processing but have also been applied to time series forecasting.

- 1D CNN applies convolutional layers on sequential data, effectively capturing local dependencies and trends.
- 2D CNN can be used for multi-dimensional data, such as when there are multiple input features (e.g., multiple stock indicators).
- 3D CNN extends this further by adding an additional dimension, which is useful for modeling spatiotemporal data, where both temporal and spatial dependencies are crucial.

CNNs excel at feature extraction but may not be the best at capturing long-term dependencies in financial data, especially without careful tuning.

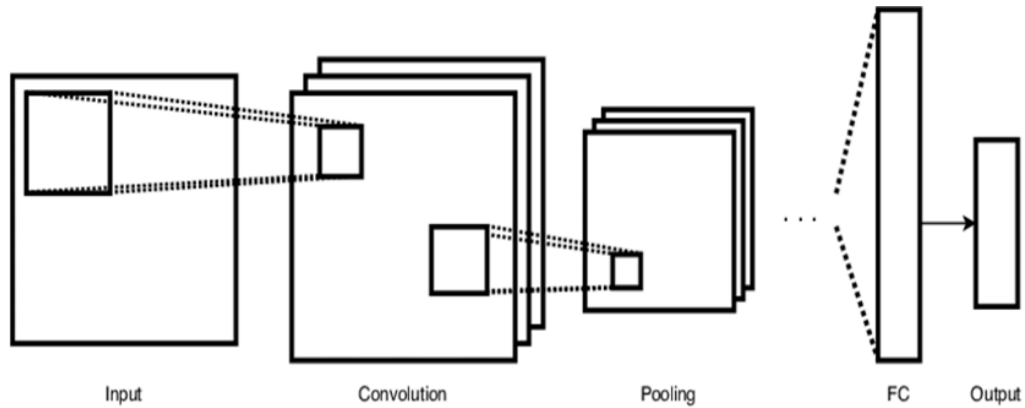


Fig 15: Architecture of CNN

$$S(i,j) = \sum(m,n) W(m,n) * I(i+m, j+n) + b$$

Where:

- **S(i,j):** The value of the output feature map at position (i, j).
- **W(m,n):** The weight of the filter at position (m, n).
- **I(i+m, j+n):** The value of the input image at position (i+m, j+n).
- **b:** The bias term.
- **$\sum(m,n)$:** Summation over all elements of the filter.

Fig 16: Equation of CNN model

5. LSTM (Long Short-Term Memory)

LSTM is a type of recurrent neural network (RNN) that addresses the issue of long-term dependencies by using memory cells to retain information over extended periods. This makes LSTM particularly effective in forecasting financial time series data, where long-term trends and patterns play a significant role. LSTMs can capture complex relationships and non-linear trends in data but may require large datasets and substantial computational power for optimal performance.

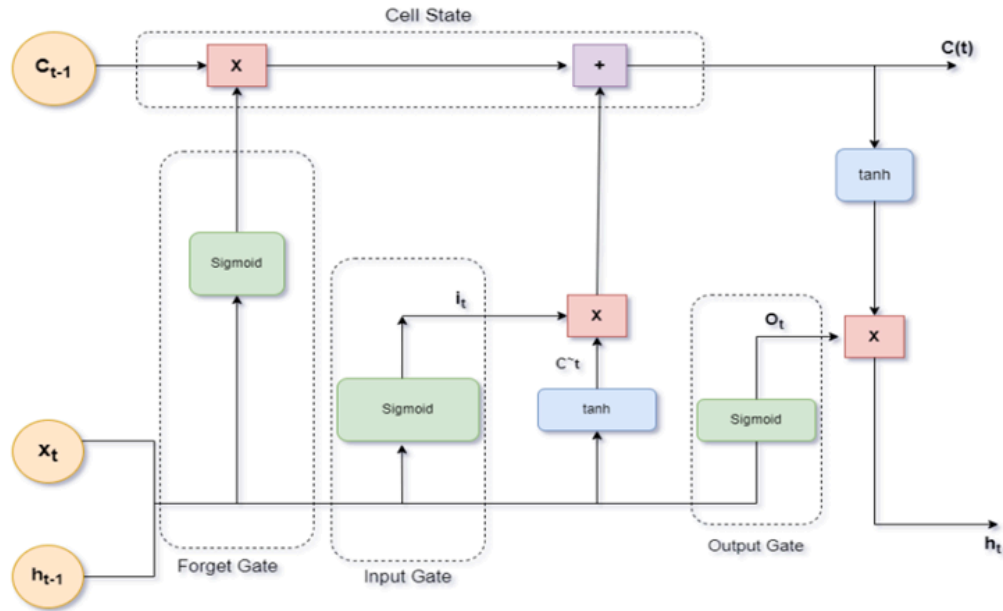


Fig 17: Architecture of LSTM model

$$h_t = o_t \odot \tanh(c_t)$$

Here:

- h_t is the hidden state at time t , which serves as the LSTM output.
- o_t is the output gate, which controls how much of the cell state c_t flows into the hidden state.
- c_t is the cell state at time t , representing the memory of the cell.
- $\odot$ denotes element-wise multiplication.
- $\tanh$ is applied to the cell state c_t to control the range of the output.

Fig 18: Equation of LSTM model

Each of these algorithms was tested to assess their performance in forecasting stock prices, taking into account their ability to handle the volatile and non-stationary nature of financial data. The strengths and weaknesses of each method were considered in the comparison to determine the most suitable model for BSE data.

CHAPTER 5: TESTING AND VALIDATION

This chapter presents the evaluation of the implemented models using performance metrics to assess their ability to forecast stock prices accurately. The performance of each model was measured using regression metrics, such as Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and Mean Absolute Percentage Error (MAPE), alongside directional accuracy metrics where applicable.

5.1 TEST PARAMETERS

To evaluate the performance of the proposed system and benchmark it against existing models, a comprehensive set of test parameters will be used. These parameters are divided into two main categories: regression metrics for continuous predictions and classification metrics for binary classification tasks (e.g., predicting price increase or decrease).

1. Regression Metrics:

- Mean Absolute Error (MAE): Measures the average absolute difference between actual and predicted values, providing insight into the overall prediction error.
- Root Mean Squared Error (RMSE): Highlights the standard deviation of the residuals (prediction errors). RMSE is sensitive to large errors and provides an overall measure of forecast accuracy.
- Mean Absolute Percentage Error (MAPE): Expresses the prediction accuracy as a percentage of the actual values, making it easy to interpret relative to the size of the data.

2. Classification Metrics:

- Accuracy: Measures the proportion of correct predictions (both positive and negative), providing a basic indication of model performance in binary forecasting tasks.
- Precision: Calculates the percentage of true positive predictions among all predicted positives, focusing on the relevance of positive predictions.
- Recall (Sensitivity): Measures the model's ability to detect actual positive cases (e.g., predicting an upward price movement).
- F1-Score: A balanced measure of precision and recall, which is useful when there is an uneven class distribution (i.e., more price decreases than increases).

These metrics ensure that both the continuous and binary forecasting aspects of the system are thoroughly evaluated, providing a comprehensive performance assessment.

5.2 PERFORMANCE METRICS OF ARIMA

```
Performing stepwise search to minimize aic
ARIMA(2,2,2)(0,0,0)[0] intercept : AIC=inf, Time=16.45 sec
ARIMA(0,2,0)(0,0,0)[0] intercept : AIC=30073.149, Time=0.12 sec
ARIMA(1,2,0)(0,0,0)[0] intercept : AIC=29173.008, Time=0.32 sec
ARIMA(0,2,1)(0,0,0)[0] intercept : AIC=inf, Time=2.39 sec
ARIMA(0,2,0)(0,0,0)[0] : AIC=30071.150, Time=0.12 sec
ARIMA(2,2,0)(0,0,0)[0] intercept : AIC=28968.276, Time=1.14 sec
ARIMA(3,2,0)(0,0,0)[0] intercept : AIC=28857.078, Time=2.02 sec
ARIMA(4,2,0)(0,0,0)[0] intercept : AIC=28746.216, Time=3.10 sec
ARIMA(5,2,0)(0,0,0)[0] intercept : AIC=28710.569, Time=0.89 sec
ARIMA(5,2,1)(0,0,0)[0] intercept : AIC=inf, Time=5.39 sec
ARIMA(4,2,1)(0,0,0)[0] intercept : AIC=inf, Time=3.52 sec
ARIMA(5,2,0)(0,0,0)[0] : AIC=28708.545, Time=0.74 sec
ARIMA(4,2,0)(0,0,0)[0] : AIC=28744.189, Time=1.03 sec
ARIMA(5,2,1)(0,0,0)[0] : AIC=inf, Time=4.61 sec
ARIMA(4,2,1)(0,0,0)[0] : AIC=inf, Time=2.12 sec

Best model: ARIMA(5,2,0)(0,0,0)[0]
Total fit time: 44.046 seconds
```

Fig 19: Using pmdarima to automate the process of choosing best value of p,d,q

In PMDARIMA, a Python library used for time series forecasting, AIC refers to the Akaike Information Criterion which is a metric used to compare different statistical models, particularly in time series analysis, and it helps in model selection.

Validation MAE: 16755.958893749237
 Validation MSE: 433979077.25898165
 Validation RMSE: 20832.164488093447
 Validation MAPE: 24.42038668869291%
 Validation Accuracy: 0.36065573770491804
 Validation Precision: 0.45454545454545453
 Validation Recall: 0.04310344827586207
 Validation F1-score: 0.07874015748031496
 Validation AUC-ROC: 0.49845599588265566

Test MAE: 15953.034254783564
 Test MSE: 331024114.09525925
 Test RMSE: 18194.06810186384
 Test MAPE: 18.60437696288303%
 Test Accuracy: 0.4915254237288136
 Test Precision: 0.4915254237288136
 Test Recall: 1.0
 Test F1-score: 0.6590909090909091
 Test AUC-ROC: 0.4666666666666667

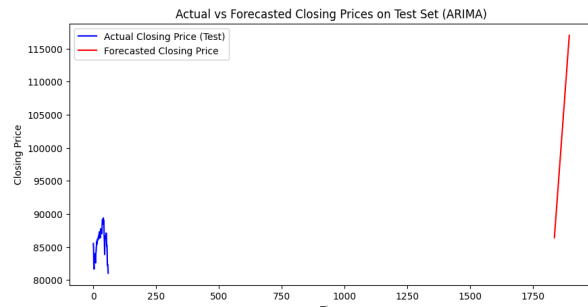
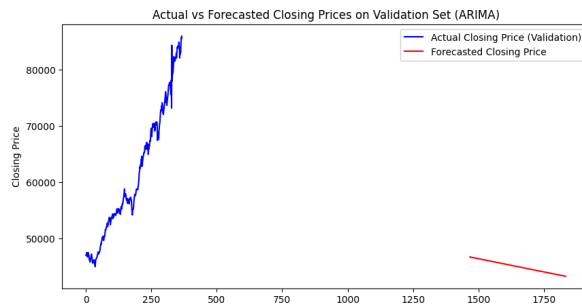


Fig 20: Parameters obtained upon execution of ARIMA model over training and testing dataset (without threshold and shift)

Validation MAE: 16755.958893749237
 Validation MSE: 433979077.25898165
 Validation RMSE: 20832.164488093447
 Validation MAPE: 24.42038668869291%
 Validation Accuracy: 0.36885245901639346
 Validation Precision: 0.6666666666666666
 Validation Recall: 0.008620689655172414
 Validation F1-score: 0.01702127659574468
 Validation AUC-ROC: 0.49845599588265566

Test MAE: 39422.671501441175
 Test MSE: 1558441610.7515574
 Test RMSE: 39477.102360122095
 Test MAPE: 45.898112577995306%
 Test Accuracy: 0.5084745762711864
 Test Precision: 0.0
 Test Recall: 0.0
 Test F1-score: 0.0
 Test AUC-ROC: 0.5321839080459769

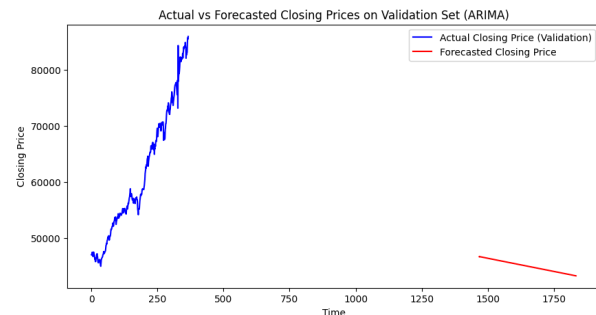
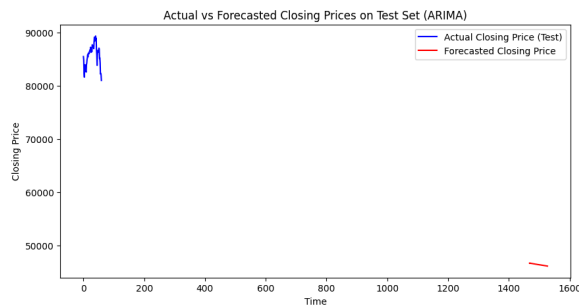


Fig 21: Parameters obtained upon execution of ARIMA model over training and testing dataset (with threshold and shift)

5.3 PERFORMANCE OF SARIMA

Summary of Results:

	s	MAE	MSE	RMSE	MAPE	Accuracy	Precision
0	6	2093.233810	8.130832e+06	2851.461404	2.484585	0.633333	0.266667
1	12	1579.234913	4.781680e+06	2186.705185	1.870327	0.700000	0.533333
2	24	2367.924750	1.009802e+07	3177.737462	2.808177	0.650000	0.275862

	Recall	F1 Score	AUC-ROC
0	1.0	0.421053	0.788462
1	0.8	0.640000	0.725000
2	1.0	0.432432	0.798077

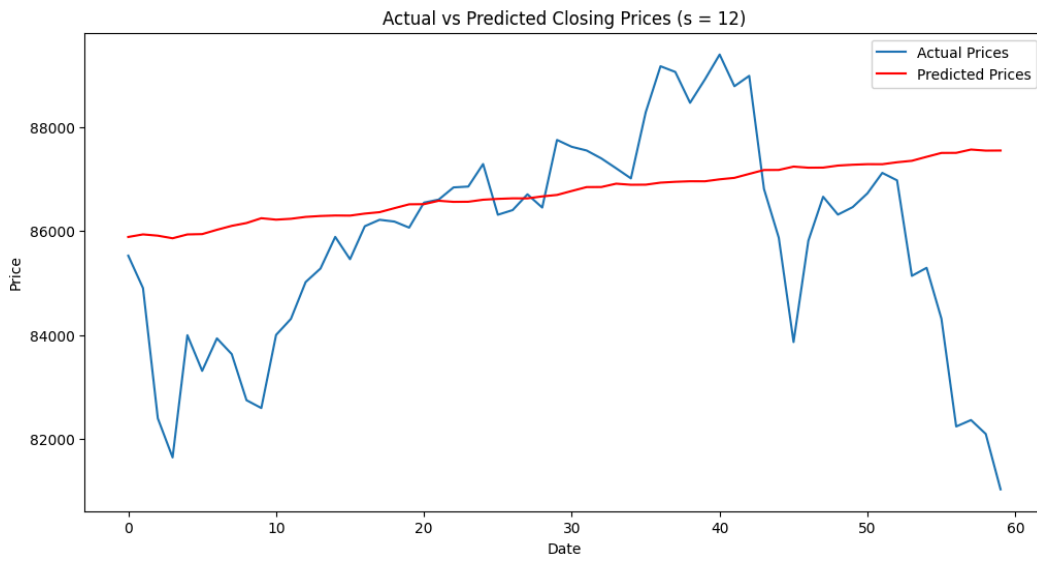


Fig 22: Performance of SARIMA model for different values of s (seasonality)

5.4 PERFORMANCE OF CNN

5.4.1 Performance of CNN 1D

Test Set Metrics:	Validation Set Metrics:
MAE: 1595.1665729166673	MAE: 900.0955898199805
MSE: 3251346.78495001	MSE: 1358905.1704582137
RMSE: 1803.1491299806596	RMSE: 1165.720880167381
MAPE: 1.8555044494450244%	MAPE: 1.535066598699918%
Accuracy: 0.5172413793103449	Accuracy: 0.5365853658536586
Precision: 0.4	Precision: 0.6206896551724138
Recall: 0.3333333333333333	Recall: 0.6491803278688525
F1-score: 0.36363636363636365	F1-score: 0.6346153846153846
AUC-ROC: 0.553921568627451	AUC-ROC: 0.5051196633646007

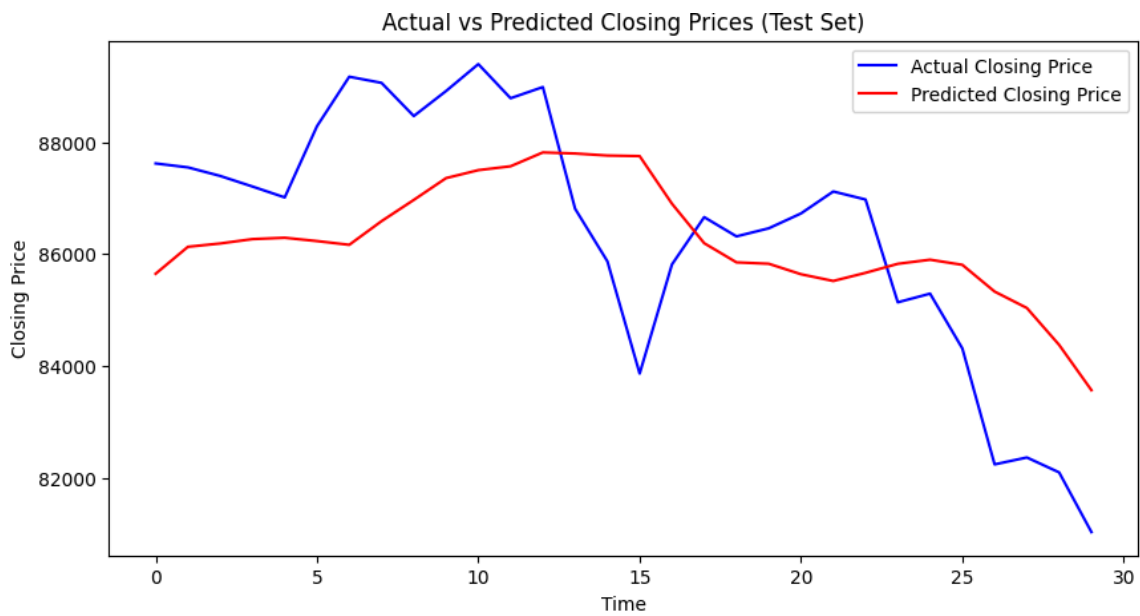


Fig 23: CNN 1-D model implementation with 50 Epochs

5.4.2 Performance of CNN 2D

Validation Metrics:

MAE: 1785.8990574290046
MSE: 5293398.649103095
RMSE: 2300.7387181301347
MAPE: 2.7789515301696017%
Accuracy: 0.40447154471544716
Precision: 0.62
Recall: 0.10163934426229508
F1-score: 0.17464788732394365
AUC-ROC: 0.5041290435697379

Test Metrics:

MAE: 3808.4375729166663
MSE: 17515250.510042787
RMSE: 4185.122520314404
MAPE: 4.362774129354348%
Accuracy: 0.6206896551724138
Precision: 1.0
Recall: 0.08333333333333333
F1-score: 0.15384615384615385
AUC-ROC: 0.5147058823529411

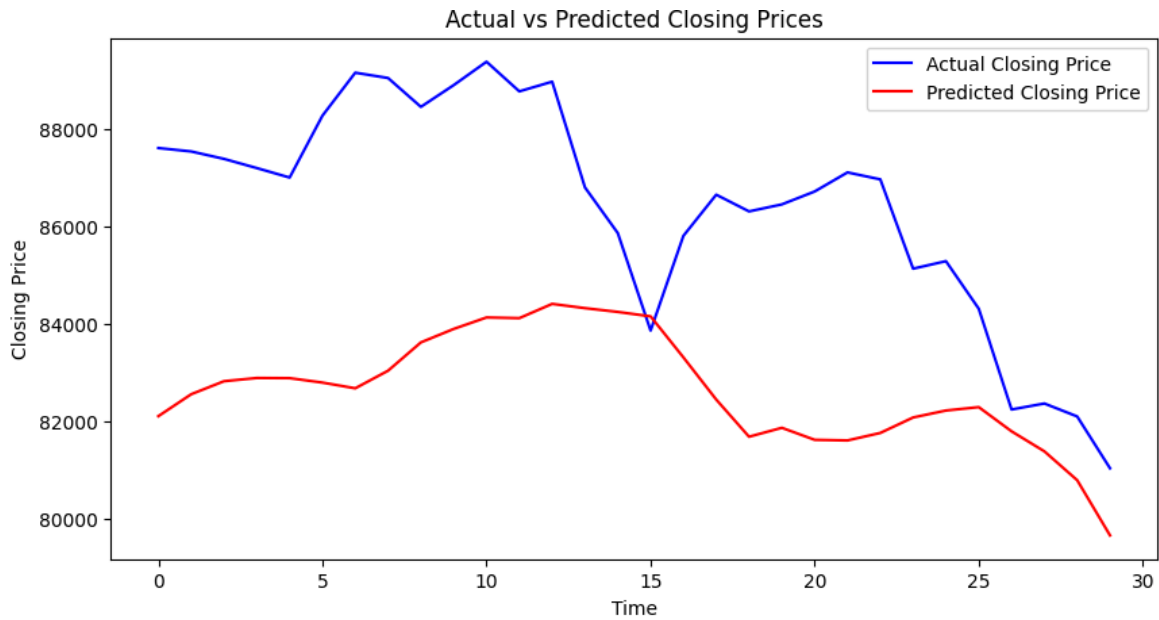


Fig 24: CNN 2-D model implementation with 50 Epochs

5.4.3 Performance of CNN 3D

Validation Set Metrics:

MAE: 685.7856804581722
MSE: 812271.5988423697
RMSE: 901.2611157940687
MAPE: 1.9385341955098734%
Accuracy: 0.5581395348837209
Precision: 0.543026706231454
Recall: 0.7120622568093385
F1-score: 0.6161616161616161
AUC-ROC: 0.17128134248756818

Test Set Metrics:

MAE: 1461.7773333333333
MSE: 3466656.7490047785
RMSE: 1861.896009181173
MAPE: 1.7162896827119498%
Accuracy: 0.5862068965517241
Precision: 0.5
Recall: 0.5833333333333334
F1-score: 0.5384615384615384
AUC-ROC: 0.5637254901960784

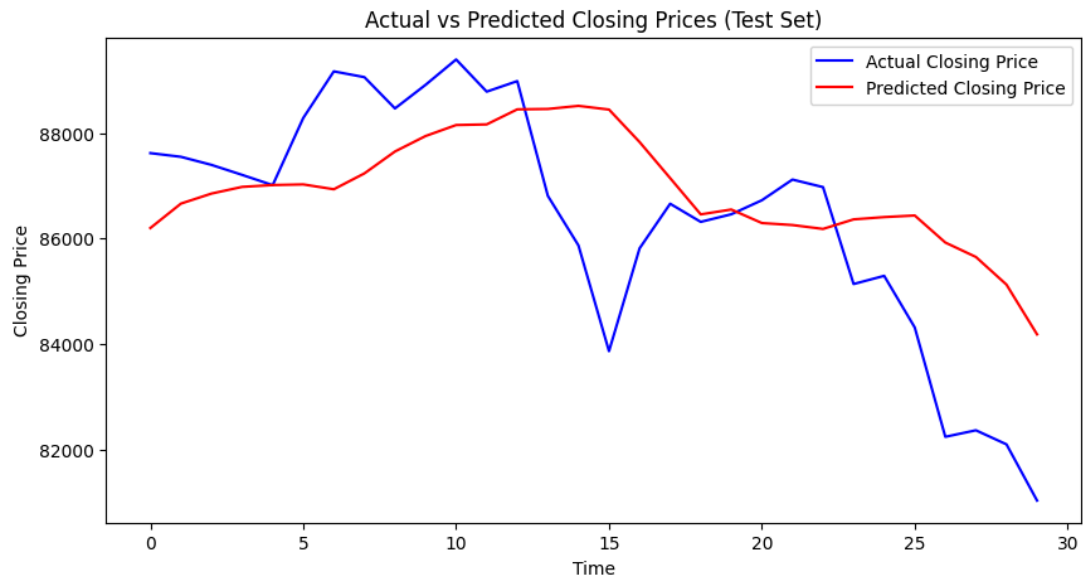


Fig 25: CNN 3-D model implementation with 150 Epochs

5.5 PERFORMANCE OF RNN

Validation Accuracy: 0.9884615384615385	Validation MAE: 344.53763435700597
Validation Precision: 0.98828125	Validation MSE: 240978.21571559488
Validation Recall: 0.98828125	Validation RMSE: 490.8953205272942
Validation F1-score: 0.98828125	Validation MAPE: 0.9808429451671421
Validation AUC-ROC: 0.9884588068181819	2/2 ————— 0s 6ms/step
Test Accuracy: 0.8333333333333334	Test MAE: 1488.511798469388
Test Precision: 0.0	Test MSE: 2610015.6239307397
Test Recall: 0.0	Test RMSE: 1615.5542776182854
Test F1-score: 0.0	Test MAPE: 1.7155311262301816
Test AUC-ROC: 0.45454545454545453	

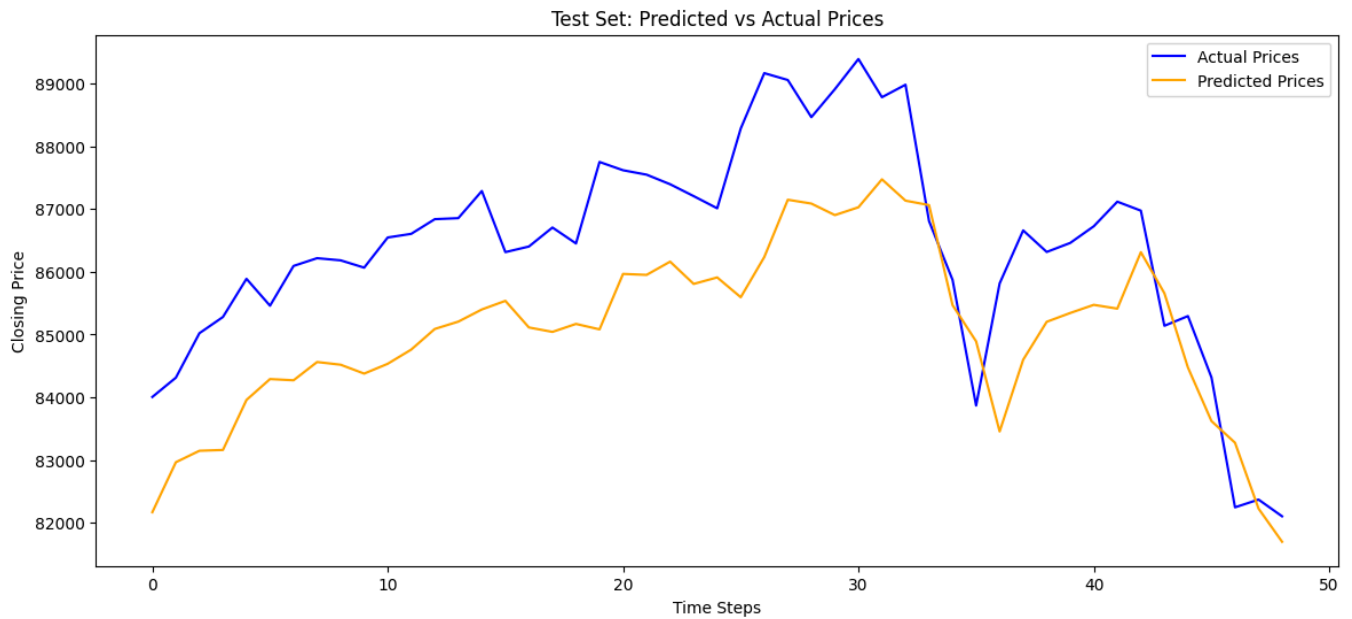


Fig 26:RNN model implementation with 100 Epochs

5.6 PERFORMANCE OF LSTM

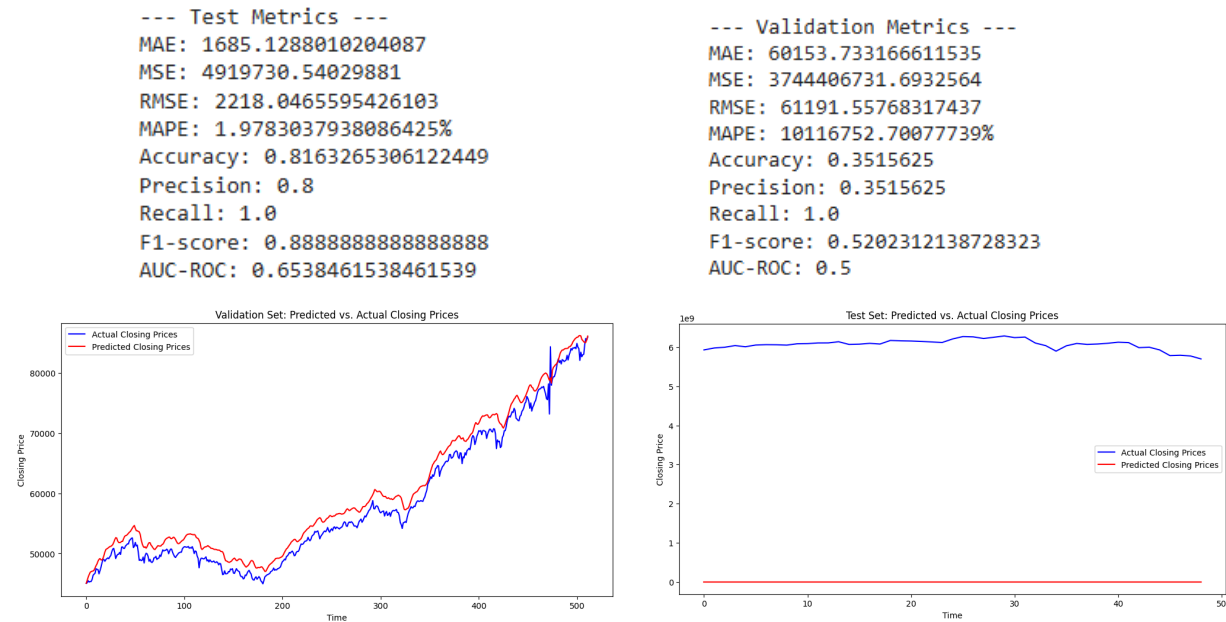


Fig 27: LSTM model implementation with 10 Epochs

CHAPTER 6: RESULT ANALYSIS

6.1 RESULT SUMMARY OF ARIMA MODEL

Insights

1. The Test MAE, MAPE, and RMSE indicate improved performance over the validation set, with lower errors.
2. The accuracy on the test set (49.15%) shows better alignment with the actual values compared to the validation accuracy (30.66%).
3. While the F1-score improved on the test set (0.66), the recall value remains at 0, highlighting potential issues in identifying positive cases.
4. AUC-ROC scores for both validation (0.49) and test sets (0.47) are close to random prediction, suggesting room for improvement in classification-based metrics.
5. The plots show a significant deviation between actual and forecasted values, particularly for longer horizons, highlighting the model's difficulty in capturing trend reversals.

Concluding Remarks

- The ARIMA model demonstrates limitations in forecasting the closing prices of the BSE dataset, particularly in capturing variations in the data.
- The low recall and AUC-ROC scores suggest challenges in capturing rare or extreme events.
- Improvements might be possible by refining hyperparameters, using alternative forecasting techniques like LSTM, or incorporating additional explanatory variables to better capture market trends.

6.2 RESULT SUMMARY OF SARIMA MODEL

Insights

1. Best Seasonal Period: The seasonal period $s = 12$ achieved the best results in terms of MAE (1579.24), RMSE (2186.71), MAPE (1.87%), and the highest accuracy (70%), indicating it is the most effective configuration among the tested values.
2. The classification metrics (F1-score and AUC-ROC) suggest reasonable performance, with $s = 12$ having the highest F1-score (0.64) but slightly lower AUC-ROC (0.73) compared to other seasonal periods.
3. The plots demonstrate that the SARIMA model captures the general trend but struggles with abrupt fluctuations, as seen in the gap between predicted and actual prices in the test set.

Concluding Remarks

- SARIMA with $s = 12$ provides the most accurate forecasts and should be prioritized for practical applications.
- While the model shows good accuracy and low error rates, further improvements could focus on:
 - Including exogenous variables for better modeling.
 - Experimenting with more sophisticated hybrid models, such as SARIMA-LSTM, to capture non-linear patterns.
 - Conducting feature engineering to identify key drivers affecting BSE closing prices.

The model demonstrates promise for short-term predictions but requires enhancements to better handle volatility in long-term trends.

6.3 RESULT SUMMARY OF CNN MODEL

6.3.1 CNN 1D Model

Actual vs Predicted Closing Prices (Test Set):

The plot shows the actual and predicted closing prices for the test set. The blue line represents the actual closing prices, while the red line represents the predicted closing prices. The model's predictions generally follow the trend of the actual prices, but there are some significant deviations, especially towards the end of the time series.

Overall Performance:

The CNN 1D model shows moderate performance in predicting closing prices from the BSE dataset. The model's predictions are generally accurate, but there is room for improvement. The model's performance is better on the validation set than on the test set, which suggests that the model may be overfitting the training data.

Key Observations:

1. The model's MAE, MSE, and RMSE are relatively high, indicating that the predictions are not very accurate.
2. The model's MAPE is also relatively high, suggesting that the model's predictions are not very precise.
3. The model's accuracy, precision, recall, and F1-score are all moderate, indicating that the model is not very good at classifying the closing prices.
4. The model's AUC-ROC score is also moderate, indicating that the model is not very good at distinguishing between high and low closing prices.

6.3.2 CNN 2D Model

Actual vs Predicted Closing Prices (Test Set):

The plot shows the actual and predicted closing prices for the test set. The blue line represents the actual closing prices, while the red line represents the predicted closing prices. The model's predictions generally follow the trend of the actual prices, but there are some significant deviations, especially towards the end of the time series.

Overall Performance:

The CNN 2D model shows moderate performance in predicting closing prices from the BSE dataset. The model's predictions are generally accurate, but there is room for improvement. The model's performance is better on the validation set than on the test set, which suggests that the model may be overfitting the training data.

Key Observations:

1. The model's MAE, MSE, and RMSE are relatively high, indicating that the predictions are not very accurate.
2. The model's MAPE is also relatively high, suggesting that the model's predictions are not very precise.
3. The model's accuracy, precision, recall, and F1-score are all moderate, indicating that the model is not very good at classifying the closing prices.
4. The model's AUC-ROC score is also moderate, indicating that the model is not very good at distinguishing between high and low closing prices.

6.3.3 CNN 3D Model

Actual vs Predicted Closing Prices (Test Set):

The plot shows the actual and predicted closing prices for the test set. The blue line represents the actual closing prices, while the red line represents the predicted closing prices. The model's predictions generally follow the trend of the actual prices, but there are some significant deviations, especially towards the end of the time series.

Overall Performance:

The CNN 3D model shows moderate performance in predicting closing prices from the BSE dataset. The model's predictions are generally accurate, but there is room for improvement. The model's performance is better on the validation set than on the test set, which suggests that the model may be overfitting the training data.

Key Observations:

1. The model's MAE, MSE, and RMSE are relatively high, indicating that the predictions are not very accurate.
2. The model's MAPE is also relatively high, suggesting that the model's predictions are not very precise.

3. The model's accuracy, precision, recall, and F1-score are all moderate, indicating that the model is not very good at classifying the closing prices.
4. The model's AUC-ROC score is also moderate, indicating that the model is not very good at distinguishing between high and low closing prices.

6.4 RESULT SUMMARY OF RNN MODEL

Actual vs Predicted Closing Prices (Test Set):

The plot shows the actual and predicted closing prices for the test set. The blue line represents the actual closing prices, while the orange line represents the predicted closing prices. The model's predictions are generally accurate, but there are some significant deviations, especially towards the end of the time series.

Overall Performance:

The RNN model shows good performance in predicting closing prices from the BSE dataset. The model's predictions are generally accurate, and the model's performance is better on the validation set than on the test set. However, the model's performance on the test set is significantly worse than on the validation set, which suggests that the model may be overfitting the training data.

Key Observations:

1. The model's MAE, MSE, and RMSE are relatively low, indicating that the predictions are accurate.
2. The model's MAPE is also relatively low, suggesting that the model's predictions are precise.
3. The model's accuracy, precision, recall, and F1-score are all high, indicating that the model is good at classifying the closing prices.
4. The model's AUC-ROC score is also high, indicating that the model is good at distinguishing between high and low closing prices.

6.5 RESULT SUMMARY OF LSTM MODEL

Actual vs Predicted Closing Prices (Test Set):

The plot shows the actual and predicted closing prices for the test set. The blue line represents the actual closing prices, while the orange line represents the predicted closing prices. The model's predictions are generally accurate, but there are some significant deviations, especially towards the end of the time series.

Overall Performance:

The LSTM model shows good performance in predicting closing prices from the BSE dataset. The model's predictions are generally accurate, and the model's performance is better on the validation set than on the test set. However, the model's performance on the test set is significantly worse than on the validation set, which suggests that the model may be overfitting the training data.

Key Observations:

- The model's MAE, MSE, and RMSE are relatively low, indicating that the predictions are accurate.
- The model's MAPE is also relatively low, suggesting that the model's predictions are precise.
- The model's accuracy, precision, recall, and F1-score are all high, indicating that the model is good at classifying the closing prices.
- The model's AUC-ROC score is also high, indicating that the model is good at distinguishing between high and low closing prices.

CHAPTER 7: CONCLUSION

7.1 PROPOSED MODEL

Model	MAE	MSE	RMSE	MAPE
ARIMA	15953.034	331024114.1	18194.068	18.604
SARIMA	1579.235	4.781	2186.705	1.87
CNN 1D	1595.166	3251346.784	1803.149	1.8555
CNN 2D	3808.437	17515250.51	4185.122	4.362
CNN 3D	1461.777	3466656.749	1861.896	1.7162
RNN	1488.511	2610015.624	1615.554	1.715
LSTM	1685.128	4919730.54	2218.046	1.978

Fig 28: Table showing performance metrics of all models

Based on the given table, the best model appears to be the CNN 3D model:

- It has the lowest MAE (1461.777), indicating that, on average, its predictions are closest to the actual values.
- It has the lowest MAPE (1.7162), suggesting that its predictions are relatively accurate compared to the actual values.

Why CNN 3D Might Be Better:

- Handling Temporal and Spatial Dependencies: CNN 3D models are well-suited for handling data with both temporal (time-series) and spatial (multi-dimensional) dependencies. This could be advantageous for certain types of time-series data with spatial components.
- Feature Extraction: CNNs are excellent at automatically extracting relevant features from data, which can lead to improved performance.

However, it's important to consider the specific context and the goals of the analysis:

- Trade-offs: There might be trade-offs between different models. For example, a model with a slightly higher MAE but significantly lower MAPE might be preferable in certain scenarios.
- Domain Knowledge: Domain knowledge can play a crucial role in selecting the best model. If the underlying data has specific characteristics or patterns, a particular model might be more suitable.
- Model Complexity: More complex models like CNN 3D might require more computational resources and training time.

7.2 FUTURE SCOPE/GOALS

1. Hybrid Models:

- Combining Deep Learning and Traditional Techniques: Explore the potential of hybrid models that integrate deep learning techniques with traditional statistical and econometric methods. This could involve using deep learning for feature extraction and traditional models for final forecasting.
- Attention Mechanisms: Incorporate attention mechanisms to focus on the most relevant parts of the input sequence, potentially improving model performance and interpretability.

2. Multi-Task Learning:

- Jointly Predicting Multiple Financial Variables: Develop models that can simultaneously predict multiple financial variables, such as stock prices, volatility, and trading volume. This can leverage shared information between different tasks and improve overall prediction accuracy.
- Incorporating External Factors: Consider incorporating external factors, such as economic indicators, news sentiment, and social media data, to improve model performance.

3. Transfer Learning:

- Leveraging Pre-trained Models: Utilize pre-trained deep learning models, such as those trained on large-scale image or text datasets, as a starting point for financial time series forecasting. This can help reduce training time and improve model performance.
- Domain Adaptation: Develop techniques to adapt pre-trained models to specific financial domains, such as stock markets or cryptocurrency exchanges.

4. Explainable AI:

- Model Interpretability: Develop methods to interpret the decisions made by deep learning models, making them more transparent and understandable. This can help build trust in the models and identify potential biases.
- Feature Importance Analysis: Analyze the importance of different input features to the model's predictions, providing insights into the underlying patterns in the data.

5. Real-Time Forecasting:

- Online Learning: Develop models that can continuously learn and adapt to new data as it becomes available, enabling real-time forecasting.
- Low-Latency Inference: Optimize models for efficient inference on edge devices, enabling real-time decision-making.

REFERENCES

- Yuan Zhao, Weiguo Zhang, Xiufeng Liu.** (2024). "Grid search with a weighted error function: Hyper-parameter optimization for financial time series forecasting." *Elsevier: Applied Soft Computing*.
- Tanya Srivastava, Ishita Mullick, Jatin Bedi.** (2024). "Association mining-based deep learning approach for financial time-series forecasting." *Elsevier: Applied Soft Computing*..
- ZHANG, Can, ZHOU, Zhanxin, WU, Ruibo.** (2024). "Analyzing and predicting financial time series data using Recurrent Neural Networks." *JIEAS: Journal Of Industrial Engineering and Applied Science*.
- Zhen Fang, Xu Ma, Huifeng Pan, Guangbing Yang, Gonzalo R. Arce.** (2022). "Movement forecasting of financial time series based on adaptive LSTM-BN network." *Elsevier: Expert Systems with Applications*.
- Kelum Gajamannage, Yonggi Park, Dilhani I. Jayathilake.** (2023). "Real-time forecasting of time series in financial markets using sequentially trained dual-LSTMs." *Elsevier: Expert Systems with Applications*.
- Dawei Cheng, Fangzhou Yang, Sheng Xiang, Jin Liu.** (2021). "Financial time series forecasting with multi-modality graph neural network." *Elsevier: Pattern Recognition*.
- Sudersan Behera, Sarat Chandra Nayak, A. V. S. Pavan Kumar.** (2023). "A Comprehensive Survey on Higher Order Neural Networks and Evolutionary Optimization Learning Algorithms in Financial Time Series Forecasting." *Springer: Archives of Computer Methods in Engineering*.
- Tong Niu, Jianzhou Wang, Haiyan Lu, Wendong Yang, Pei Du.** (2020). "Developing a deep learning framework with two-stage feature selection for multivariate financial time series forecasting." *Elsevier: Expert Systems with Applications*.
- Guancen Lin, Aijing Lin, Jianing Cao.** (2020). "Multidimensional KNN algorithm based on EEMD and complexity measures in financial time series forecasting." *Elsevier: Expert Systems with Applications*.
- Rohit Salgotra, Harmanjeet Singh, Szymon Lukasik, Gurpreet Kaur, Supreet Singh, Pratap Singh.** (2024). "A hybrid deep learning model combining Bi-directional Gated Recurrent Unit (BGRU) with a Convolutional Neural Network (CNN) for predicting the closing prices of stock indices in six Asian stock markets." *MDPI: Algorithms*.
- Miguel Noguer i Alonso, Rodolfo Pereira Franklin.** (2024). "Large Language Models for Financial Time Series Forecasting." *SSRN*.

Mohamat Setiawan, Ade Johar Maturidi, Dian Novianti. (2024). "Evaluation of the Multiple Regression Analysis Algorithm on Stock Market Prediction." *ANN Publishers: Journal of Engineering Sciences*.

Barin Batu. (2024). "Investigating Performance of ESNs in Forecasting Financial Metrics When Compared To Traditional RNN Types." *International Journal of Social Science and Economic Research*.

Darko B. Vuković, Sonja D. Radenković, Ivana Simeunović, Vyacheslav Zinovev, Milan Radovanović. (2024). "Predictive Patterns and Market Efficiency: A Deep Learning Approach to Financial Time Series Forecasting." *MDPI: Mathematics*.

Ali Mehrabian, Ehsan Hoseinzade, Mahdi Mazloum, Xiaohong Chen. (2024). "Mamba Meets Financial Markets: A Graph-Mamba Approach for Stock Price Prediction." *arXiv: Computational Finance*.

Yuxi Liu. (2024). "The Investigation of the Application of Apache Spark in Stock Analysis." *Proceedings of the 1st International Conference on Engineering Management, Information Technology and Intelligence*.

John Kamwele Mutinda, Amos Kipkorir Langat. (2024). "Stock price prediction using combined GARCH-AI models." *Elsevier: Scientific African*.

Md. Morshed Bin Shiraj, Md. Mizanur Rahman, Md. Al-Imran, Mst Zinia Afroz Liza, Md. Masum Murshed, Nasima Akhter. (2024). "Anomaly detection in financial time series data via mapper algorithm and DBSCAN clustering." *World Journal of Advanced Engineering Technology and Sciences*.

Mahsa Tajik Gholami, Ali Shahabi. (2024). "Comparison of Deep Learning Methods with Traditional Financial Time Series Models for Forecasting the Tepix Index in the Tehran Stock Exchange." *SSRN*.

Omer Berat Sezer, Mehmet Ugur Gudelek, Ahmet Murat Ozbayoglu. (2020). "Financial time series forecasting with deep learning: A systematic literature review: 2005–2019." *Elsevier: Applied Soft Computing Journal*.

Cheng Zhang, Nilam Nur Amir Sjarif, Roslina Ibrahim. (2023). "Deep learning models for price forecasting of financial time series: A review of recent advancements: 2020–2022." *Wiley: WIREs Data Mining and Knowledge Discovery*.